

Shakti’s Algorithm: Efficient Enumeration of Bounded Circuits Using Set-Trie Filtering

Shakti Singh
Independent Researcher

May 28, 2026

Abstract

Enumerating all valid cycles in a directed graph is a fundamental problem in graph theory. While established algorithms efficiently discover simple cycles, extracting non-simple circuits—where nodes may repeat but edges are bounded by capacity constraints—introduces severe combinatorial explosion. In this paper, we introduce Shakti’s Algorithm, a novel Depth-First Search architecture optimized for circuit enumeration. Furthermore, we address the $O(C^2 \cdot M)$ bottleneck of filtering subsumed topological sub-cycles by introducing a highly cache-friendly Set-Trie optimization, drastically reducing time complexity. Empirical evaluations in C++ demonstrate significant performance gains over naive subset filtering on dense graphs.

1 Introduction

The enumeration of cycles within directed graphs has widespread applications in network analysis, deadlock detection, and bounded model checking. Traditionally, algorithms have focused strictly on simple cycles, halting execution upon vertex repetition to prevent infinite loops. However, in applications requiring complex closed walks where edges—rather than vertices—are bounded by capacity, traditional algorithms fail. This paper presents Shakti’s Algorithm, an architecture designed specifically to traverse, enumerate, and filter these bounded circuits efficiently.

2 Preliminaries and Related Work

While Johnson’s algorithm [1] remains the gold standard for simple cycle enumeration, bounding its execution to $O((V + E)(C + 1))$, it explicitly ignores non-simple paths. Approaches using Eulerian trail combinations can extract circuits but struggle to handle partial edge repetition efficiently. Shakti’s Algorithm bridges this gap using edge-tracked Depth-First Search with lexicographical pruning.

3 Shakti’s Algorithm: Core Architecture

Our approach divides the traversal space into an entry anchor and an edge-tracked frontier. By mapping "visited" states to directed edges rather than vertices, the algorithm permits vertex self-intersection seamlessly.

4 Set-Trie Optimization

Generating raw circuits is only the first phase. Topological sub-cycles (circuits completely subsumed within the edge-footprint of larger paths) must be filtered. A naive pairwise comparison

Algorithm 1 Shakti's Algorithm (Base DFS Generator)

```
1: procedure EXPLOREFRONTIER( $u, entry\_node$ )
2:   for each edge  $e \in adj[u]$  do
3:     if  $edge\_visits[e.id] \geq MAX\_VISITS$  then
4:       continue
5:     end if
6:     if  $e.to < entry\_node$  then
7:       continue ▷ Lexicographical Pruning
8:     end if
9:      $edge\_visits[e.id] \leftarrow edge\_visits[e.id] + 1$ 
10:     $current\_path.push\_back(e.to)$ 
11:    if  $e.to == entry\_node$  then
12:       $SaveCircuit(current\_path, edge\_visits)$ 
13:    end if
14:    EXPLOREFRONTIER( $e.to, entry\_node$ )
15:     $edge\_visits[e.id] \leftarrow edge\_visits[e.id] - 1$  ▷ Backtrack
16:     $current\_path.pop\_back()$ 
17:  end for
18: end procedure
```

requires $O(C^2 \cdot M)$ time, causing severe bottlenecks. We resolve this by projecting edge footprints as sorted multisets into a Superset-Trie. By evaluating generated circuits in descending order of length, the Trie permits $O(C \cdot \text{Trie Depth})$ filtering, instantly pruning search branches via early-exit edge comparisons.

5 Proof of Correctness

Let $G = (V, E)$ be a directed graph. We define a *bounded circuit* C as a sequence of vertices $v_0, v_1, \dots, v_p$ where $v_0 = v_p$, and for any directed edge $e \in E$, the frequency of e traversed in C does not exceed a predefined capacity k .

Definition 1 (Lexicographical Minimum). *For any bounded circuit C , let v_{min} denote the vertex in C with the minimum integer identifier. The canonical representation of C is the cyclic permutation that begins and ends with v_{min} .*

Lemma 1 (Termination). *Shakti's Algorithm terminates for any finite directed graph $G = (V, E)$.*

Proof. The Depth-First Search maintains a frequency counter for each edge $e \in E$. An edge is only traversed if its current frequency is strictly less than k . Therefore, the absolute maximum length of any traversal path is bounded by $k \cdot |E|$. Since both the number of edges and the capacity k are finite, the depth of the recursion tree is strictly bounded, guaranteeing termination. $\square$

Lemma 2 (Soundness). *Every sequence output by the DFS generator is a valid bounded circuit.*

Proof. A sequence is recorded if and only if the current vertex equals the *entry_node*. Since the traversal always originates at *entry_node*, the recorded path forms a closed walk $v_0 \dots v_p$ where $v_0 = v_p$. Furthermore, the recursion condition $edge_visits[e] < k$ strictly enforces the edge capacity constraint. Thus, the output is a valid bounded circuit. $\square$

Lemma 3 (Completeness). *Every valid bounded circuit C in G is discovered by the algorithm exactly once.*

Proof. Let C be a valid bounded circuit with a unique minimum vertex v_{min} . The algorithm's outer loop iterates through all vertices $v \in V$ as the *entry_node*. When *entry_node* = v_{min} , the algorithm begins a DFS from v_{min} . Because C is valid, all vertices $v_i \in C$ satisfy $v_i \geq v_{min}$. Therefore, the pruning condition ($v_i < \text{entry_node}$) is never triggered during the traversal. As the DFS exhaustively explores all unpruned branches via backtracking, C is guaranteed to be found. Conversely, if the outer loop sets *entry_node* = v_j where $v_j \in C$ but $v_j > v_{min}$, the traversal will eventually encounter v_{min} . At this point, the pruning condition ($v_{min} < \text{entry_node}$) correctly aborts the branch, preventing C from being recorded as a shifted duplicate. $\square$

Theorem 1 (Correctness of Shakti's Algorithm). *The algorithm correctly enumerates all maximal bounded circuits in G .*

Proof. By Lemmas 1, 2, and 3, the generation phase correctly produces the exhaustive set of bounded circuits. Let $F(C)$ represent the edge footprint of circuit C . A circuit B is a topological sub-cycle of A if and only if $F(B) \subseteq F(A)$. The Set-Trie processes all circuits in descending order of length. The Trie traversal matches the sorted elements of $F(B)$ against the Trie branches. The search prunes a branch if and only if the Trie edge is strictly greater than the required edge $e \in F(B)$. Because the Trie children are monotonically sorted, this perfect decision guarantees that only maximal circuits are retained. $\square$

6 Complexity Analysis

Let $N = |V|$ be the number of vertices and $M = |E|$ be the number of directed edges in graph G . Let k denote the maximum permitted traversals per edge. Let C_{raw} be the total number of bounded circuits generated, and C_{max} be the subset of maximal circuits.

6.1 Phase 1: Bounded DFS Generation

Time Complexity: In the absolute worst-case scenario, the Depth-First Search can traverse every edge up to k times, resulting in a maximum recursion depth of $k \cdot M$. The upper bound of the search tree is $O(N \cdot \Delta^{k \cdot M})$, where Δ is the maximum out-degree of G . However, the lexicographical pruning rule ensures that branches are terminated immediately if they violate the *entry_node* minimum. The time taken to output the discovered circuits is bounded by $O(C_{raw} \cdot k \cdot M)$. Therefore, the time complexity is strictly proportional to the number of valid bounded paths explored rather than the theoretical maximum topological space.

Space Complexity: The spatial overhead during traversal is highly minimal. The algorithm requires $O(M)$ auxiliary space for the *edge_visits* frequency array and $O(k \cdot M)$ space for the *current_path* recursion stack. Storing the generated circuits in memory requires $O(C_{raw} \cdot \bar{L})$, where $\bar{L} \leq k \cdot M$ is the average circuit length. Total space complexity for the generation phase is $O(M + C_{raw} \cdot \bar{L})$.

6.2 Phase 2: Set-Trie Filtering

Time Complexity: A naive pairwise topological subset filter requires $O(C_{raw}^2 \cdot M)$ time, effectively reducing the problem to a Time Limit Exceeded (TLE) state for large outputs. The Set-Trie optimization bypasses this bottleneck. Sorting the circuits by length requires $O(C_{raw} \log C_{raw})$ time. Querying and inserting into the Set-Trie requires $O(D_{Trie} \cdot \Delta_{Trie})$ operations per circuit, where $D_{Trie} \leq k \cdot M$ is the maximum depth of the Trie, and Δ_{Trie} is the branching factor at each node. By pruning non-viable superset branches instantly, the filtering time complexity is reduced from quadratic to $O(C_{raw} \log C_{raw} + C_{raw} \cdot D_{Trie})$.

Space Complexity: The Set-Trie constructs branches only for the maximal circuits that survive the filter. In the worst case, the Trie requires $O(C_{max} \cdot k \cdot M)$ nodes. By utilizing a flat

array memory pool instead of fragmented heap allocations, the spatial footprint remains highly compact and cache-friendly.

7 Empirical Evaluation

To validate the theoretical bounds and memory efficiency of Shakti’s Algorithm, it was benchmarked against the highly skewed **web-Google** directed graph provided by the Stanford Network Analysis Project (SNAP). The test was conducted using a custom zero-overhead Compressed Sparse Row (CSR) parser on standard consumer hardware (24GB DDR4 Memory) with a bounded capacity limit of $C = 5$.

Table 1: Performance Metrics on SNAP **web-Google** Dataset

Dataset Specification	Value
Nodes	916,428
Edges	5,105,039
Performance Metrics ($C = 5$)	
Total Sub-Cycles Evaluated	311,730,081
Unique Topological Sets Filtered	66,181,920
Execution Time (Seconds)	435.39
Throughput (Cycles/Sec)	$\approx 715,000$

7.1 Discussion of Results

The algorithm successfully evaluated over 311 million cycles while allocating and filtering over 66 million unique Set-Trie nodes in main memory without triggering out-of-memory constraints. This empirical throughput demonstrates that the hardware-aware CSR array traversal, when combined with the Set-Trie topological filtering, effectively bypasses the severe $O(C^2 \cdot M)$ subsumption bottleneck typically encountered in dense, real-world graphs.

8 Conclusion

We have presented Shakti’s Algorithm, a highly optimized C++ architecture for the enumeration of non-simple circuits under bounded edge capacity. By replacing naive subset comparisons with a Set-Trie, the algorithm bypasses traditional $O(C^2)$ bottlenecks, offering a robust tool for advanced graph-theoretic analysis.

References

- [1] Donald B Johnson. Finding all the elementary circuits of a directed graph. *SIAM Journal on Computing*, 4(1):77–84, 1975.